

Fast tokenizers-এর বিশেষ ক্ষমতা

এই অংশে আমরা **Transformers**-এর tokenizer-এর বাড়তি ক্ষমতাগুলো দেখব।

এখন পর্যন্ত tokenizer আমরা মূলত ২টি কাজের জন্য ব্যবহার করেছি:

- input text কে token এ ভাঙা
- token ID থেকে আবার text বের করা

কিন্তু tokenizer, বিশেষ করে যেগুলো **Tokenizers** লাইব্রেরির ওপর তৈরি, সেগুলো আরও অনেক বেশি কাজ করতে পারে।

এই extra feature বোঝানোর জন্য এখানে দেখানো হচ্ছে কীভাবে **token-classification (NER)** আর **question-answering** pipeline-এর result manually বের করা যায়।

Slow tokenizer আর Fast tokenizer কী?

এখানে ২ ধরনের tokenizer বলা হয়েছে:

Slow tokenizer

- **Transformers** লাইব্রেরির ভেতরে Python দিয়ে লেখা

Fast tokenizer

- **Tokenizers** লাইব্রেরির মাধ্যমে দেওয়া হয়
- Rust দিয়ে লেখা
- তাই অনেক দ্রুত কাজ করে

Fast tokenizer কেন fast?

নিচের টেবিলে fast আর slow tokenizer-এর speed comparison দেখানো হয়েছে:

Fast tokenizer	Slow tokenizer
batched=True → 10.8s	4min 41s
batched=False → 59.2s	5min 3s

মানে, অনেক text একসাথে tokenize করলে fast tokenizer অনেক দ্রুত হয়।

গুরুত্বপূর্ণ কথা

একটা মাত্র sentence tokenize করলে সবসময় fast tokenizer noticeably দ্রুত নাও হতে পারে। কখনও কখনও slow tokenizer-এর মতোই হতে পারে, এমনকি একটু slow-ও লাগতে পারে।

কিন্তু অনেকগুলো text একসাথে tokenize করলে fast tokenizer-এর advantage পরিষ্কার বোঝা যায়।

Batch encoding কী?

Tokenizer-এর output সাধারণ Python dictionary না।

এটা আসলে একটি **BatchEncoding object**।

এটা dictionary-এর মতো behave করে, তাই আগের মতো indexing করা যায়। তবে fast tokenizer-এর জন্য এতে extra কিছু method থাকে।

Fast tokenizer-এর সবচেয়ে important feature

Fast tokenizer-এর সবচেয়ে গুরুত্বপূর্ণ ক্ষমতা হলো:

এরা সবসময় track রাখে যে final token আসলে original text-এর কোন জায়গা থেকে এসেছে।

এই feature-কে বলে **offset mapping**।

এর ফলে আমরা জানতে পারি:

- কোন word থেকে কোন token তৈরি হয়েছে
- original text-এর কোন character কোন token-এর মধ্যে আছে
- আবার কোন token text-এর কোন character range-এর সাথে সম্পর্কিত

উদাহরণ

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")  
example = "My name is Sylvain and I work at Hugging Face in Brooklyn."  
encoding = tokenizer(example)  
print(type(encoding))
```

এখানে output হবে:

```
<class 'transformers.tokenization_utils_base.BatchEncoding'>
```

মানে tokenizer আমাদের `BatchEncoding` object দিয়েছে।

Fast tokenizer কিনা কীভাবে বুঝবে?

দুইভাবে check করা যায়।

1) tokenizer থেকে

`tokenizer.is_fast`

Output:

True

2) encoding থেকে

`encoding.is_fast`

Output:

True

Fast tokenizer দিয়ে কী কী করা যায়?

1) সরাসরি token দেখা যায়

`encoding.tokens()`

Output:

```
[[CLS], 'My', 'name', 'is', 'S', '##yl', '##va', '##in', 'and', 'I', 'work', 'at', 'Hu', '##gging', 'Face', 'in', 'Brooklyn', '.', '[SEP]']
```

এখানে দেখা যাচ্ছে "`Sylvain`" শব্দটি ভেঙে কয়েকটি token হয়েছে:

- `S`
- `##yl`
- `##va`
- `##in`

2) কোন token কোন word থেকে এসেছে তা জানা যায়

`encoding.word_ids()`

Output:

[None, 0, 1, 2, 3, 3, 3, 3, 4, 5, 6, 7, 8, 8, 9, 10, 11, 12, None]

এটার মানে:

- `[CLS]` আর `[SEP]` special token, তাই এদের জন্য `None`
- বাকি token কোন word থেকে এসেছে, তার index দেখাচ্ছে

যেমন:

- `"Sylvain"` sentence-এর word index 3
- তাই `S, ##yl, ##va, ##in` — সবগুলোর word id হলো 3

এটা কেন useful?

এটা দিয়ে বোঝা যায়:

- token কোনো word-এর শুরুতে আছে কিনা
- দুইটা token একই word-এর অংশ কিনা

আগে BERT-এর `##` prefix দেখে আন্দাজ করা যেত, কিন্তু সেটা শুধু BERT-এর মতো tokenizer-এ কাজ করে।

`word_ids()` method যেকোনো **fast tokenizer**-এ কাজ করে।

এটা খুব দরকারি:

- **NER**
- **POS tagging**
- **whole word masking**

এর মতো task-এ।

Word আসলে কী — এটা সবসময় **same** না

একটা word কী, সেটা tokenizer-এর উপর depend করে।

উদাহরণ:

`I' ll`

এটা কি ১টা word, নাকি ২টা word?

এটা tokenizer-এর pre-tokenization rule-এর উপর depend করবে।

কিছু tokenizer

- শুধু space দেখে split করে
- তাহলে "I' ll" = ১টা word

কিছু tokenizer

- punctuation-ও দেখে split করে
- তাহলে "I' ll" = ২টা word

Practice idea

bert-base-cased আর roberta-base tokenizer দিয়ে "81s" tokenize করলে পার্থক্য দেখা যেতে পারে।

দেখার বিষয়:

- token কীভাবে split হলো
- word IDs কী আসলো

sentence_ids() method

আরেকটা method আছে:

- sentence_ids()

এটা বলে দেয় token কোন sentence থেকে এসেছে।

যদিও pair input-এর ক্ষেত্রে tokenizer-এর token_type_ids দিয়েও similar info পাওয়া যায়।

Word বা token থেকে original text-এর character position বের করা

Fast tokenizer দিয়ে আমরা original text-এর character range-ও বের করতে পারি।

Useful methodগুলো:

- word_to_chars()

- `token_to_chars()`
- `char_to_word()`
- `char_to_token()`

উদাহরণ

```
start, end = encoding.word_to_chars(3)
example[start:end]
```

Output:

Sylvain

এখানে `word id = 3` কোন `text span`-এর মধ্যে আছে, সেটা বের করা হয়েছে।

মানে "`Sylvain`" original sentence-এর কোন `character range`-এ আছে, সেটা পাওয়া গেছে।

এটা সম্ভব হচ্ছে কেন?

কারণ fast tokenizer প্রতিটি token-এর জন্য original text-এর **offset** track করে রাখে।

এই offset list-এর মাধ্যমেই আমরা token আর original text-এর relation বুঝতে পারি।

বাস্তব কাজে এটা কোথায় লাগে?

এই feature ব্যবহার করে manually pipeline-এর result reproduce করা যায়, যেমন:

- token classification
- named entity recognition
- question answering

সহজ ভাষায় পুরো বিষয়টার সারাংশ

Fast tokenizer-এর ৩টা বড় সুবিধা:

1) Speed

অনেক text একসাথে tokenize করলে অনেক দ্রুত

2) Token-to-word mapping

কোন token কোন word থেকে এসেছে সেটা জানা যায়

3) Character span mapping

Original text-এর কোন অংশ থেকে token এসেছে সেটা জানা যায়

Inside the token-classification pipeline

Chapter 1-এ আমরা প্রথমবার **NER (Named Entity Recognition)** ব্যবহার দেখেছিলাম।

NER কী?

NER হলো এমন একটি task যেখানে text-এর কোন অংশ:

- Person
- Location
- Organization

এসব entity বোঝায়, সেটা খুঁজে বের করা হয়।

উদাহরণ:

- Sylvain → Person
- Hugging Face → Organization
- Brooklyn → Location

pipeline() কী কাজ করে?

Chapter 2-এ দেখানো হয়েছিল যে `pipeline()` raw text থেকে prediction বের করার জন্য ৩টি step একসাথে করে:

1) Tokenization

Text কে token-এ ভাগ করা

2) Model-এ পাঠানো

Tokenized input model-এর মধ্যে দেওয়া

3) Post-processing

Model-এর raw output কে readable result-এ রূপান্তর করা

এখানে বলা হচ্ছে:

- প্রথম ২টা step অন্য pipeline-এর মতোই
- কিন্তু **token-classification pipeline**-এর **post-processing** একটু বেশি **complex**

কারণ এখানে শুধু prediction বের করলেই হয় না, token গুলোকে entity হিসেবে ঠিকভাবে interpret-ও করতে হয়।

Pipeline দিয়ে basic result বের করা

প্রথমে একটি token classification pipeline নেওয়া হচ্ছে:

```
from transformers import pipeline
```

```
token_classifier = pipeline("token-classification")
```

```
token_classifier("My name is Sylvain and I work at Hugging Face in Brooklyn.")
```

এখানে default model হলো:

`dbmdz/bert-large-cased-finetuned-conll03-english`

এই model sentence-এর উপর NER করে।

Output কী দেখাচ্ছে?

Output:

```
[{'entity': 'I-PER', 'score': 0.9993828, 'index': 4, 'word': 'S', 'start': 11, 'end': 12},  
{ 'entity': 'I-PER', 'score': 0.99815476, 'index': 5, 'word': '##yl', 'start': 12, 'end': 14},  
{ 'entity': 'I-PER', 'score': 0.99590725, 'index': 6, 'word': '##va', 'start': 14, 'end': 16},  
{ 'entity': 'I-PER', 'score': 0.9992327, 'index': 7, 'word': '##in', 'start': 16, 'end': 18},  
{ 'entity': 'I-ORG', 'score': 0.97389334, 'index': 12, 'word': 'Hu', 'start': 33, 'end': 35},  
{ 'entity': 'I-ORG', 'score': 0.976115, 'index': 13, 'word': '##gging', 'start': 35, 'end': 40},  
{ 'entity': 'I-ORG', 'score': 0.98879766, 'index': 14, 'word': 'Face', 'start': 41, 'end': 45},  
{ 'entity': 'I-LOC', 'score': 0.99321055, 'index': 16, 'word': 'Brooklyn', 'start': 49, 'end': 57}]
```

এই output-এর মানে কী?

Model প্রতিটি token-এর জন্য আলাদা prediction দিয়েছে।

"Sylvain" এর ক্ষেত্রে

শব্দটা কয়েকটি token-এ split হয়েছে:

- S
- ##yl
- ##va
- ##in

এবং সব token-কে I-PER বলা হয়েছে।

মানে:

- এগুলো **Person entity**-এর অংশ

"Hugging Face" এর ক্ষেত্রে

- Hu
- ##gging
- Face

এগুলোকে I-ORG বলা হয়েছে।

মানে:

- এগুলো **Organization**-এর অংশ

"Brooklyn" এর ক্ষেত্রে

- Brooklyn token-টিকে I-LOC বলা হয়েছে

মানে:

- এটা **Location**

Output-এর fieldগুলো কী বোঝায়?

প্রতিটি item-এ কয়েকটি জিনিস আছে:

entity

কোন ধরনের entity

- **I-PER** = Person entity-এর অংশ
- **I-ORG** = Organization entity-এর অংশ
- **I-LOC** = Location entity-এর অংশ

score

Model কতটা confident

index

Token-এর position

word

Token নিজে

start, end

Original text-এর character position

যেমন:

- **Sylvain** text-এ 11 থেকে 18 position-এর মধ্যে আছে

সমস্যা কোথায়?

এখানে output token level-এ এসেছে।

অর্থাৎ:

- **Sylvain**-কে একসাথে না দেখিয়ে ৪টা token হিসেবে দেখাচ্ছে
- **Hugging Face**-কেও আলাদা token হিসেবে দেখাচ্ছে

কিন্তু আমরা সাধারণত final result চাই word/entity level-এ।

Token গুলোকে group করা

এই জন্য pipeline-এ **aggregation_strategy="simple"** ব্যবহার করা হয়:

```
from transformers import pipeline
```

```
token_classifier = pipeline("token-classification", aggregation_strategy="simple")
```

token_classifier("My name is Sylvain and I work at Hugging Face in Brooklyn.")

Output:

```
[{'entity_group': 'PER', 'score': 0.9981694, 'word': 'Sylvain', 'start': 11, 'end': 18},  
{ 'entity_group': 'ORG', 'score': 0.97960204, 'word': 'Hugging Face', 'start': 33, 'end': 45},  
{ 'entity_group': 'LOC', 'score': 0.99321055, 'word': 'Brooklyn', 'start': 49, 'end': 57}]
```

এখন কী হলো?

এখন token গুলো একসাথে group হয়েছে।

আগে:

- S, ##yl, ##va, ##in

এখন:

- Sylvain

আগে:

- Hu, ##gging, Face

এখন:

- Hugging Face

এটা অনেক বেশি readable।

aggregation_strategy কী?

এটা বলে দেয়, একাধিক token মিলে একটি entity হলে final score কীভাবে বের হবে।

1) "simple"

Entity-এর সব token-এর score-এর mean নেওয়া হয়।

উদাহরণ:

Sylvain-এর score = S, ##yl, ##va, ##in — এই ৪টা token-এর score-এর average

আরও **strategy** আছে

2) "**first**"

Entity-এর প্রথম token-এর score নেওয়া হয়।

উদাহরণ:

Sylvain-এর score হবে **S** token-এর score

3) "**max**"

Entity-এর যেসব token আছে, তাদের মধ্যে maximum score নেওয়া হয়।

উদাহরণ:

Hugging Face-এর score হবে তার tokenগুলোর মধ্যে সবচেয়ে বড় score

4) "**average**"

Entity-এর মধ্যে থাকা wordগুলোর score average করা হয়।

এটা "**simple**"-এর কাছাকাছি, কিন্তু একটু আলাদা ভাবে গণনা করে, বিশেষ করে যখন entity-তে একাধিক word থাকে।

"**simple**" আর "**average**"-এর পার্থক্য

এটা একটু গুরুত্বপূর্ণ।

"**simple**"

সব token-এর score average করে

"**average**"

আগে word level score বের করে, তারপর wordগুলো average করে

উদাহরণ:

Hugging Face

এখানে:

- **Hugging** কয়েকটি token থেকে তৈরি
- **Face** একটি word

"average" strategy আগে:

- **Hugging**-এর word score বের করবে
- **Face**-এর score নেবে
তারপর এই দুই word-এর গড় করবে

From inputs to predictions

এখানে দেখানো হচ্ছে raw text থেকে model কীভাবে prediction বের করে।

মূল idea হলো:

1. text tokenize করা
2. token গুলো model-এ পাঠানো
3. model-এর output logits থেকে probability আর prediction বের করা
4. তারপর কোন token কোন entity তা বোঝা

Step 1: Tokenizer আর model load করা

```
from transformers import AutoTokenizer, AutoModelForTokenClassification
```

```
model_checkpoint = "dbmdz/bert-large-cased-finetuned-conll03-english"  
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)  
model = AutoModelForTokenClassification.from_pretrained(model_checkpoint)
```

```
example = "My name is Sylvain and I work at Hugging Face in Brooklyn."  
inputs = tokenizer(example, return_tensors="pt")  
outputs = model(**inputs)
```

এখানে কী হলো?

- **AutoTokenizer** text কে token বানাল
- **AutoModelForTokenClassification** NER model load করল
- **inputs = tokenizer(...)** text কে model-এর input format-এ নিল
- **outputs = model(**inputs)** model prediction দিল

Step 2: Output shape বোঝা

```
print(inputs["input_ids"].shape)
print(outputs.logits.shape)
```

Output:

```
torch.Size([1, 19])
torch.Size([1, 19, 9])
```

এর মানে কী?

`inputs["input_ids"].shape = [1, 19]`

মানে:

- batch size = 1
- total token = 19

অর্থাৎ, ১টা sentence আছে, আর সেটা ১৯টা token-এ ভেঙেছে।

`outputs.logits.shape = [1, 19, 9]`

মানে:

- ১টা sentence
- ১৯টা token
- প্রতিটি token-এর জন্য ৯টা label-এর score

অর্থাৎ, model প্রতিটি token-এর জন্য ৯টা possible class-এর score দিচ্ছে।

Step 3: Logits থেকে probability আর prediction বের করা

import torch

```
probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)[0].tolist()
predictions = outputs.logits.argmax(dim=-1)[0].tolist()
print(predictions)
```

Output:

```
[0, 0, 0, 0, 4, 4, 4, 4, 0, 0, 0, 0, 6, 6, 6, 0, 8, 0, 0]
```

এখানে কী হচ্ছে?

Logits কী?

Model সরাসরি final label দেয় না। আগে raw score দেয়, যাকে বলে **logits**।

Softmax কী?

`softmax()` logits কে probability-তে convert করে।

মানে:

- কোন label-এর chance কত, সেটা বের করে

Argmax কী?

`argmax()` সবচেয়ে বড় score যেটার, সেই label index নেয়।

মানে:

- final prediction কোন class হবে, সেটা ঠিক করে

Step 4: Label mapping দেখা

`model.config.id2label`

Output:

```
{  
  0: 'O',  
  1: 'B-MISC',  
  2: 'I-MISC',  
  3: 'B-PER',  
  4: 'I-PER',  
  5: 'B-ORG',  
  6: 'I-ORG',  
  7: 'B-LOC',  
  8: 'I-LOC'  
}
```

এই **label** গুলোর মানে কী?

0

এটা কোনো entity না

0 = Outside

B-XXX

Entity-এর শুরু

I-XXX

Entity-এর ভেতরের অংশ

যেমন:

- B-PER = Person entity শুরু
- I-PER = Person entity-এর ভেতরের অংশ
- B-ORG = Organization শুরু
- I-ORG = Organization-এর অংশ
- B-LOC = Location শুরু
- I-LOC = Location-এর অংশ

Example অনুযায়ী কী আশা করা যায়?

ধরো "Sylvain" tokenized হয়ে হয়েছে:

- S
- ##yl
- ##va
- ##in

সাধারণভাবে আমরা ভাবতে পারি:

- S হবে B-PER
- বাকি token গুলো হবে I-PER

কিন্তু model সবগুলো token-এর জন্য I-PER দিয়েছে।

Model কি ভুল করেছে?

পুরোপুরি না।

কারণ এখানে **IOB format**-এর বিষয় আছে।

IOB1 আর IOB2

Entity labeling-এর ২টা format আছে:

IOB2

এখানে entity শুরু হলেই **B-** ব্যবহার হয়

উদাহরণ:

- **S** → **B-PER**
- **##y1** → **I-PER**

IOB1

এখানে **B-** সবসময় শুরুতে লাগে না

মূলত একই ধরনের দুইটা entity পাশাপাশি থাকলে আলাদা করতে **B-** ব্যবহার হয়

এই model যেই dataset-এ fine-tune হয়েছে, সেটা **IOB1 format** ব্যবহার করেছে।

তাই **S** token-এও **I-PER** আসতে পারে।

অর্থাৎ model-এর output dataset convention অনুযায়ী ঠিকই আছে।

Step 5: O ছাড়া useful token গুলো রাখা

```
results = []
```

```
tokens = inputs.tokens()
```

```
for idx, pred in enumerate(predictions):
```

```
    label = model.config.id2label[pred]
```

```
    if label != "O":
```

```
        results.append(
```

```
            {"entity": label, "score": probabilities[idx][pred], "word": tokens[idx]}
```

```
        )
```

```
print(results)
```

Output হবে এমন:

```
[  
    {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S'},
```

```
{'entity': 'I-PER', 'score': 0.99815476, 'word': '##yl'},
{'entity': 'I-PER', 'score': 0.99590725, 'word': '##va'},
{'entity': 'I-PER', 'score': 0.9992327, 'word': '##in'},
{'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu'},
{'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging'},
{'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face'},
{'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn'}
]
```

এখানে কী হলো?

আমরা:

- সব token দেখলাম
- যেগুলোর label 0 না, সেগুলো রাখলাম
- প্রতিটির entity, score, word নিলাম

এতে token-level NER result পাওয়া গেল।

কিন্তু এখনও কী missing?

এখানে token-এর:

- label আছে
- score আছে
- word আছে

কিন্তু original sentence-এর **start** আর **end character position** নেই।

Pipeline আগে এই তথ্যও দিয়েছিল।

এই কাজের জন্য দরকার **offset mapping**।

Step 6: Offset mapping নেওয়া

```
inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)
inputs_with_offsets["offset_mapping"]
```

Output:

```
[(0, 0), (0, 2), (3, 7), (8, 10), (11, 12), (12, 14), (14, 16), (16, 18), (19, 22), (23, 24), (25, 29), (30, 32),
```

(33, 35), (35, 40), (41, 45), (46, 48), (49, 57), (57, 58), (0, 0)]

Offset mapping কী?

প্রতিটি token original text-এর কোন character range থেকে এসেছে, সেটা tuple আকারে দেয়।

যেমন:

- (11, 12) মানে text-এর 11 থেকে 12 position
- (12, 14) মানে 12 থেকে 14 position

(0, 0) কেন?

এগুলো special token-এর জন্য, যেমন:

- [CLS]
- [SEP]

উদাহরণ

ধরো index 5 token হলো ##y1

এর offset হলো (12, 14)

example[12:14]

Output:

yl

খেয়াল করো:

- token ছিল ##y1
- কিন্তু original text slice হলো yl

মানে offset mapping token-এর original text position ঠিকভাবে ধরতে পারে।

Step 7: Final result-এ start/end যোগ করা

```
results = []
```

```
inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)
```

```
tokens = inputs_with_offsets.tokens()
offsets = inputs_with_offsets["offset_mapping"]
```

```
for idx, pred in enumerate(predictions):
    label = model.config.id2label[pred]
    if label != "O":
        start, end = offsets[idx]
        results.append(
            {
                "entity": label,
                "score": probabilities[idx][pred],
                "word": tokens[idx],
                "start": start,
                "end": end,
            }
        )
```

```
print(results)
```

Output:

```
[
    {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S', 'start': 11, 'end': 12},
    {'entity': 'I-PER', 'score': 0.99815476, 'word': '##yl', 'start': 12, 'end': 14},
    {'entity': 'I-PER', 'score': 0.99590725, 'word': '##va', 'start': 14, 'end': 16},
    {'entity': 'I-PER', 'score': 0.9992327, 'word': '##in', 'start': 16, 'end': 18},
    {'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu', 'start': 33, 'end': 35},
    {'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging', 'start': 35, 'end': 40},
    {'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face', 'start': 41, 'end': 45},
    {'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn', 'start': 49, 'end': 57}
]
```

এখন কী পাওয়া গেল?

এখন প্রতিটি entity token-এর জন্য আমরা পেলাম:

- label
- score
- token
- original text-এর start position
- original text-এর end position

এটাই pipeline-এর প্রথম result-এর মতো।

পুরো বিষয়টা সহজভাবে

আগে কী করেছি?

- text tokenize করেছি
- model-এ দিয়েছি
- logits পেয়েছি

তারপর কী করেছি?

- softmax দিয়ে probability বানিয়েছি
- argmax দিয়ে final label নিয়েছি
- `id2label` দিয়ে label-এর নাম জেনেছি
- 0 বাদ দিয়ে entity token রেখেছি
- offset mapping দিয়ে start/end position বের করেছি

সবচেয়ে important points

1) Model token ধরে predict করে

word ধরে না, token ধরে

2) Output shape [1, 19, 9]

মানে:

- 1 sentence
- 19 token
- প্রতি token-এর জন্য 9 label

3) 0 মানে কোনো entity না

4) B- আর I-

- B- = beginning
- I- = inside

5) offset mapping খুব দরকারি

কারণ এটা original sentence-এ token কোথায় ছিল সেটা বলে

From inputs to predictions

এখানে দেখানো হচ্ছে, কীভাবে একটি input sentence থেকে model step by step prediction বের করে।

Step 1: Input tokenize করা এবং model-এ পাঠানো

প্রথমে tokenizer আর model load করা হয়:

```
from transformers import AutoTokenizer, AutoModelForTokenClassification

model_checkpoint = "dbmdz/bert-large-cased-finetuned-conll03-english"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
model = AutoModelForTokenClassification.from_pretrained(model_checkpoint)

example = "My name is Sylvain and I work at Hugging Face in Brooklyn."
inputs = tokenizer(example, return_tensors="pt")
outputs = model(**inputs)
```

এখানে যা হচ্ছে:

- `AutoTokenizer` sentence-টাকে token-এ ভাঙছে
- `AutoModelForTokenClassification` সেই token গুলোর জন্য label predict করছে
- `return_tensors="pt"` মানে output PyTorch tensor আকারে আসবে

Step 2: Model output-এর shape বোঝা

```
print(inputs["input_ids"].shape)
print(outputs.logits.shape)
```

Output:

```
torch.Size([1, 19])
torch.Size([1, 19, 9])
```

এর মানে:

- batch size = 1
- total token = 19
- প্রতিটি token-এর জন্য model 9টা label-এর score দিয়েছে

অর্থাৎ model-এর output shape হলো:

1 x 19 x 9

মানে:

- 1টা sentence
- 19টা token
- প্রতি token-এর জন্য 9টা possible class

Step 3: Logits থেকে prediction বের করা

Model সরাসরি final label দেয় না। আগে raw score দেয়, যাকে বলে **logits**।

তারপর softmax ব্যবহার করে probability বের করা হয়, আর argmax ব্যবহার করে সবচেয়ে সম্ভাব্য label নেওয়া হয়।

```
import torch
```

```
probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)[0].tolist()
predictions = outputs.logits.argmax(dim=-1)[0].tolist()
print(predictions)
```

Output:

```
[0, 0, 0, 0, 4, 4, 4, 4, 0, 0, 0, 0, 6, 6, 6, 0, 8, 0, 0]
```

এখানে:

- `softmax()` raw logits কে probability-তে বদলায়
- `argmax()` সবচেয়ে বড় score-এর label index নেয়

Step 4: Label index-এর মানে বোঝা

```
model.config.id2label
```

Output:

```
{
  0: 'O',
  1: 'B-MISC',
  2: 'I-MISC',
  3: 'B-PER',
```

```
4: 'I-PER',
5: 'B-ORG',
6: 'I-ORG',
7: 'B-LOC',
8: 'I-LOC'
}
```

এখানে:

- 0 মানে token কোনো entity না
- B-XXX মানে entity-এর শুরু
- I-XXX মানে entity-এর ভেতরের অংশ

উদাহরণ:

- B-PER = person entity শুরু
- I-PER = person entity-এর ভেতর
- B-ORG = organization শুরু
- I-ORG = organization-এর অংশ
- B-LOC = location শুরু
- I-LOC = location-এর অংশ

Step 5: কেন S token-এও I-PER এসেছে?

সাধারণভাবে মনে হতে পারে:

- S হওয়া উচিত B-PER
- ##y1, ##va, ##in হওয়া উচিত I-PER

কিন্তু model S-কেও I-PER দিয়েছে।

এটা পুরোপুরি ভুল না। কারণ entity labeling-এর 2টা format আছে:

IOB2

এখানে entity শুরু হলেই B- ব্যবহার করা হয়

IOB1

এখানে B- মূলত তখন ব্যবহার হয়, যখন একই ধরনের 2টা entity পাশাপাশি থাকে এবং আলাদা করে দেখাতে হয়

এই model যেই dataset-এ fine-tune হয়েছে, সেটা IOB1 format ব্যবহার করেছে। তাই S token-এও I-PER আসতে পারে।

Step 6: 0 ছাড়া সব entity token বের করা

```
results = []
tokens = inputs.tokens()

for idx, pred in enumerate(predictions):
    label = model.config.id2label[pred]
    if label != "O":
        results.append(
            {"entity": label, "score": probabilities[idx][pred], "word": tokens[idx]}
        )

print(results)
```

Output হবে এরকম:

```
[
  {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S'},
  {'entity': 'I-PER', 'score': 0.99815476, 'word': '##y!'},
  {'entity': 'I-PER', 'score': 0.99590725, 'word': '##va'},
  {'entity': 'I-PER', 'score': 0.9992327, 'word': '##in'},
  {'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu'},
  {'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging'},
  {'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face'},
  {'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn'}
]
```

এখানে আমরা:

- সব token-এর prediction দেখেছি
- যেগুলোর label 0 না, শুধু সেগুলো রেখেছি
- তাদের entity, score আর token নিয়েছি

Step 7: Offset mapping দিয়ে start/end position বের করা

আগের result-এ একটা জিনিস missing ছিল:
token original sentence-এর কোন জায়গায় ছিল, সেটা।

এটার জন্য `return_offsets_mapping=True` ব্যবহার করা হয়।

```
inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)
inputs_with_offsets["offset_mapping"]
```

Output:

```
[(0, 0), (0, 2), (3, 7), (8, 10), (11, 12), (12, 14), (14, 16), (16, 18), (19, 22), (23, 24), (25, 29), (30, 32), (33, 35), (35, 40), (41, 45), (46, 48), (49, 57), (57, 58), (0, 0)]
```

প্রতিটি tuple দেখায় token original text-এর কোন character range থেকে এসেছে।

যেমন:

- (12, 14) মানে text-এর index 12 থেকে 14 পর্যন্ত

(0, 0) special token-এর জন্য, যেমন [CLS], [SEP]

উদাহরণ:

example[12:14]

Output:

yl

অর্থাৎ token ##yl original text-এ yl অংশের সাথে মিলে।

Step 8: Final result-এ start এবং end যোগ করা

```
results = []
inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)
tokens = inputs_with_offsets.tokens()
offsets = inputs_with_offsets["offset_mapping"]
```

```
for idx, pred in enumerate(predictions):
    label = model.config.id2label[pred]
    if label != "O":
        start, end = offsets[idx]
        results.append(
            {
                "entity": label,
                "score": probabilities[idx][pred],
                "word": tokens[idx],
                "start": start,
                "end": end,
            }
        )
```

)

```
print(results)
```

এখন output হবে:

```
[  
  {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S', 'start': 11, 'end': 12},  
  {'entity': 'I-PER', 'score': 0.99815476, 'word': '##yl', 'start': 12, 'end': 14},  
  {'entity': 'I-PER', 'score': 0.99590725, 'word': '##va', 'start': 14, 'end': 16},  
  {'entity': 'I-PER', 'score': 0.9992327, 'word': '##in', 'start': 16, 'end': 18},  
  {'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu', 'start': 33, 'end': 35},  
  {'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging', 'start': 35, 'end': 40},  
  {'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face', 'start': 41, 'end': 45},  
  {'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn', 'start': 49, 'end': 57}  
]
```

এটাই pipeline-এর প্রথম output-এর মতো result।